

Thus, messages are passed between the client, which is the UI and the host, which is the platform, using platform channels.

A standard message codec is used during the process. The platform channels support the serialisation of simple JSON like values. The serialisation and deserialisation of these values happen automatically when values are sent and received.

If one wants to use one's platform-specific code for more than one Flutter app, one can separate the code into a kind of platform plugin which is located in the directory outside the main application.

One can also publish one's platform-specific code if one wants it to be used by other Flutter developers.

Instead of using the more complex, `MethodChannel`, one can make use of the basic `BasicMessageChannel`. This supports a basic, asynchronous message passing by making use of a custom message codec. One can also use `BinaryCodec` or `StringCodec` or `JSONMessageCodec`. Alternatively, one can also create one's own codec.

Packages and Plugins

Using Packages



Flutter supports the use of shared packages which have been contributed by other developers to the Flutter and Dart ecosystem. This helps in building an application faster because one need not start from the very beginning if this is the case.

The already existing packages cater to a vast number of requirements, including making a network request, custom navigation handling integrating with device APIs, and making use of other outsider platforms like Firebase.

Flutter publishes its packages on the Pub site. Its landing page reveals the best packages which are compatible with Flutter. It supports searching among all the published packages.